# BÁO CÁO THỰC HÀNH

## Thực hành Nhập môn Mạng máy tính

## Lab 01: Mật mã học cổ điển

GVTH: Nguyễn Ngọc Trưởng

Ngày báo cáo: 19/03/2026

# Nhóm 13 – IT005.Q22.1

**THÔNG TIN CHUNG**

| MSSV | Họ và tên | Nội dung báo cáo | % công việc |
|---|---|---|---|
| 24521388 | Nguyễn Hoàng Phúc | Câu 1, câu 2, câu 3 | 50% |
| 23520536 | Phạm Cao Minh Hoàng | Câu 4, câu 5, câu 6, câu 7 | 50% |

## 1. Mật mã Caeser

### a) *Yêu cầu 2.1*

Thực hiện mã hóa và giải mã bằng ngôn ngữ C++:

1. Cấu trúc chính chương trình:
   - **Hàm caesarCipher**:
     + Nhận vào 1 chuỗi và 1 khóa k. Duyệt qua từng kí tự và dịch chuyển nó theo vòng tròn 26 chữ cái rồi trả về chuỗi mới.
     + Hàm tính key với cấu trúc an toàn, kể cả trong trường hợp người dùng nhập vào 1 số âm, công thức sẽ chuyển đổi nó thành số dương tương ứng trên vòng tròn 26 chữ cái Ví dụ K = -3 ⇔ K = 23. Tránh chương trình bị lỗi khi xử lí.
     + "*String result = "";*" : Đây là biến rỗng để lưu trữ các chữ cái sau bước giải mã
     + Vòng lặp sau đó sẽ duyệt qua từng kí tự có trong chuỗi vừa nhập, kiểm tra loại kí tự in hoa/ in thường với cấu trúc if / else if. Trong trường hợp kí tự in hoa, phép toán "*result += char((c - 'A' + key) % 26 + 'A');*" sẽ làm từng bước các việc như đưa chữ cái về số thứ tự từ 0 đến 25, cộng thêm key, lấy phần dư với phép chia cho 26 để đảm bào nằm trong phạm vi bảng chữ cái rồi ép con số đó trở thành mã chữ cái in hoa với cú pháp "+ 'A'" và đẩy vào result. Áp dụng tương tự cho trường hợp chữ thường, trường hợp còn lại là của số/ kí tự đặc biệt.

```cpp
string caesarCipher(string text, int key) {
    key = (key % 26 + 26) % 26;
    string result = "";
    for (char c : text) {
        if (c >= 'A' && c <= 'Z') {
            result += char((c - 'A' + key) % 26 + 'A');
        }
        else if (c >= 'a' && c <= 'z') {
            result += char((c - 'a' + key) % 26 + 'a');
        }
        else {
            result += c;
        }
    }
    return result;
}
```

*Hình 1.1 Hàm caesarCipher*

   - **Hàm bruteForce:**
     + Nhận vào 1 đoạn văn bản mã hóa để tính toán.
     + Khởi động vòng lặp quét nghiệm bắt đầu từ 1, kết thúc ở 25.
     + "*string plaintext = caesarCipher(ciphertext, 26 - key);*": Gọi lại hàm caesarCipher để giải mã, truyền vào khóa là 26 – key để "tiến" về đúng vị trí giải mã. Kết quả lưu vào biến plaintext.
     + "*string lowerText = ""; for (char c : plaintext) { lowerText += tolower(c); }*": Tạo ra 1 chuỗi lowerText chứa nội dung thử dưới dạng chữ in thường.

+ "*if (lowerText.find(" the ") != string::npos)*": Đây là cốt lõi của hàm – nhận diện từ tiếng anh thường được sử dụng, ở đây em sử dụng từ "the", có thể thêm các từ thông dụng khác nếu muốn chương chạy hiệu quả hơn. Nếu tìm được sẽ ngay lập tức in ra màn hình khóa K và văn bản hoàn chỉnh. Ngược lại sẽ xuất ra thông báo không tìm thấy.

```
#include <iostream>
#include <string>

using namespace std;

string caesarCipher(string text, int key) {
    key = (key % 26 + 26) % 26;
    string result = "";
    for (char c : text) {
        if (c >= 'A' && c <= 'Z') {
            result += char((c - 'A' + key) % 26 + 'A');
        }
        else if (c >= 'a' && c <= 'z') {
            result += char((c - 'a' + key) % 26 + 'a');
        }
        else {
            result += c;
        }
    }
    return result;
}

void bruteForce(string ciphertext) {
    for (int key = 1; key <= 25; key++) {
        string plaintext = caesarCipher(ciphertext, 26 - key);
        string lowerText = "";
        for (char c : plaintext) {
            lowerText += tolower(c);
        }

        if (lowerText.find(" the ") != string::npos) {
            cout << "Khoa K = " << key << "\n";
            cout << "Ban ro: \n" << plaintext << "\n";
            return;
        }
    }
    cout << "Khong tim thay ket qua phu hop!\n";
}

```

```cpp
int main() {
    int choice;
    cout << "1. Ma hoa\n2. Giai ma\n3. Brute-force\nChon chuc nang (1/2/3): ";
    cin >> choice;
    cin.ignore();

    if (choice == 1 || choice == 2) {
        string text;
        int key;
        cout << "Nhap chuoi van ban: ";
        getline(cin, text);
        cout << "Nhap khoa K: ";
        cin >> key;

        if (choice == 2) {
            key = 26 - (key % 26);
        }
        cout << "Ket qua: " << caesarCipher(text, key) << "\n";

    }
    else if (choice == 3) {
        string ciphertext;
        cout << "Nhap ban ma can Brute-force: \n";
        getline(cin, ciphertext);
        bruteForce(ciphertext);
    }

    return 0;
}
```

*Hình 1.2 Source code đầy đủ*

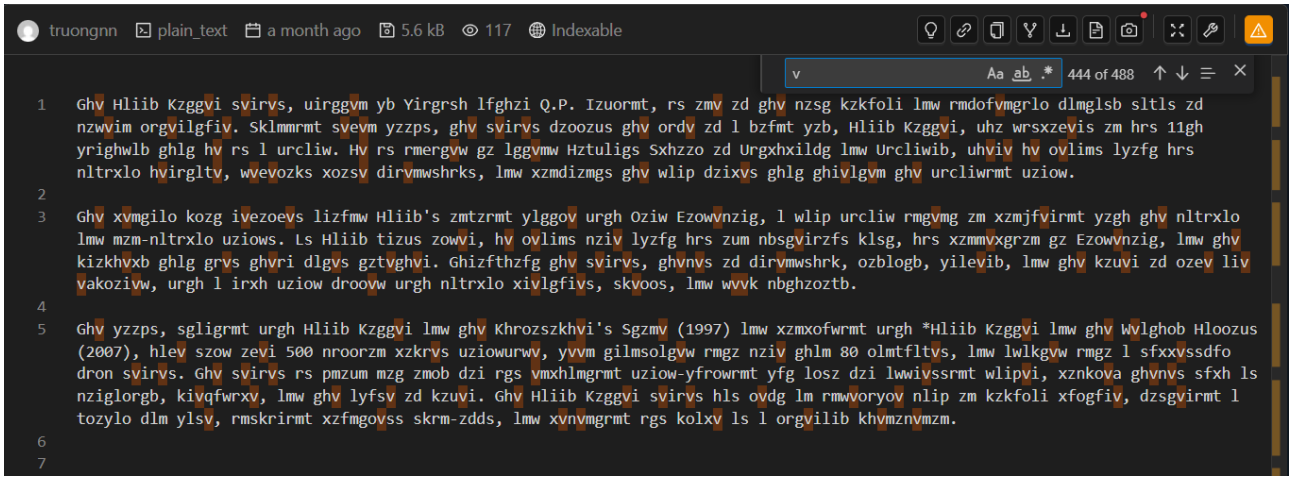
```
Microsoft Visual Studio Debug Console
1. Ma hoa
2. Giai ma
3. Brute-force
Chon chuc nang (1/2/3): 3
Nhap ban ma can Brute-force:
Max NBM bl t extwbgz bglmbmnmbhg ngwxk OGN-AVF, lixvbtebsbgz bg max ybxew hy bgyhkftmbhg mxvaghehzr. Xlmtueblaxw pbma t
fbllbhg mh yhlmxk bgghotmbhg tgw xqvxeexgvx bg BM xwnvtmbhg tgw kxlxtkva, NBM hyyxkl t pbwx ktgzx hy ngwxkzktwntmx tgw i
hlmzktwntmx ikhzktfl tbfxw tm ikhwnvbgz abzaer ldbeexw ikhyxllbhgtel. Max ngboxklbmr bl kxvhzgbsxw yhk bml vnmmbgz-xwzx
kxlxtkva bg tkxtl ebdx vruxklxvnkbmr, tkmbybvbte bgmxeebzxgvx, tgw lhymptkx xgzbgxxkbgz. Pbma lmtmx-hy-max-tkm ytvbebmbx
l tgw t lmkhgz xfiatlbl hg vheetuhktmbhg pbma bgwnlmkr, NBM xjnbil lmnwxgml pbma uhma maxhkxmbvte dghpexwzx tgw iktvmbvt
e ldbeel mh makbox bg max ktibwer xoheobgz mxva bgwnlmkr.
Khoa K = 19
Ban ro:
The UIT is a leading institution under VNU-HCM, specializing in the field of information technology. Established with a
mission to foster innovation and excellence in IT education and research, UIT offers a wide range of undergraduate and p
ostgraduate programs aimed at producing highly skilled professionals. The university is recognized for its cutting-edge
research in areas like cybersecurity, artificial intelligence, and software engineering. With state-of-the-art facilitie
s and a strong emphasis on collaboration with industry, UIT equips students with both theoretical knowledge and practica
l skills to thrive in the rapidly evolving tech industry.

C:\Users\admin\Documents\HK2_Nam2_2025-2026\NT101\TH\Lab01\Project1\x64\Debug\Project1.exe (process 3156) exited with co
de 0 (0x0).
Press any key to close this window . . .
```

*Hình 1.3 Thực hiện bruteforce khi chỉ biết ciphertext*

## 2. Mono-alphabeticSubstitutionCipherandFrequencyAnalysis

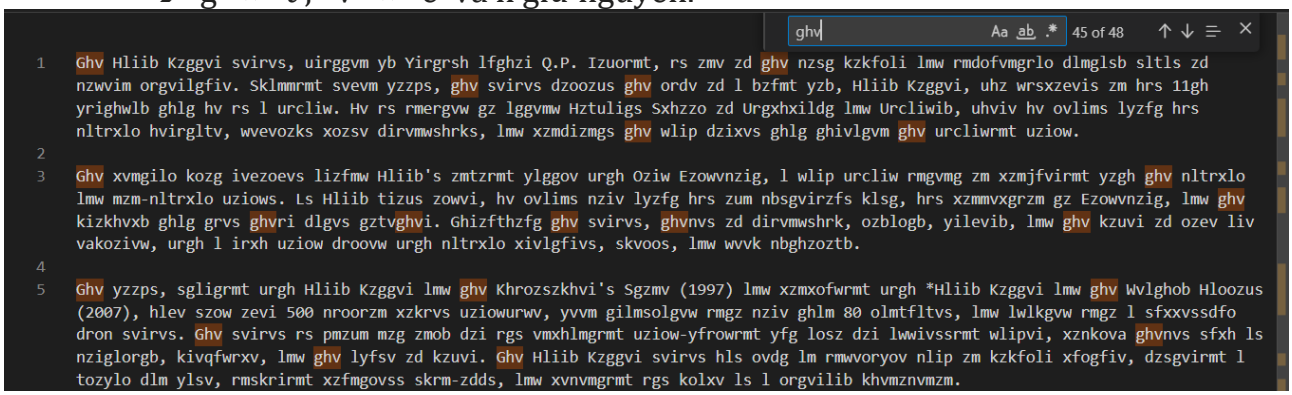
### *b) Yêu cầu 2.2*

- Trong đoạn văn bản mã hóa, chữ cái v xuất hiện nhiều nhất, trong các văn bản tiếng anh, chữ xuất hiện nhiều nhất là e, có khả năng v ⇔ e.



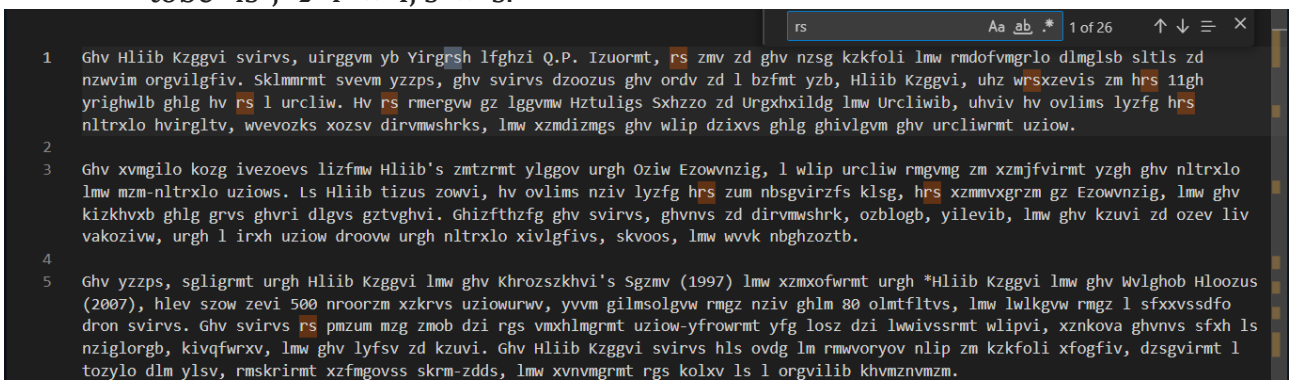

*Hình 2.1 Tần suất xuất hiện của 'v'*

- Tiếp đến là từ "ghv", từ này xuất hiện nhiều đặc biệt ở đầu cầu câu, trong số những từ có 3 chữ cái xuất hiện nhiều nhất tiếng anh từ "the" là khả thi nhất ➔ 'g' ⇔ 't'; 'v' ⇔ 'e' và h giữ nguyên.



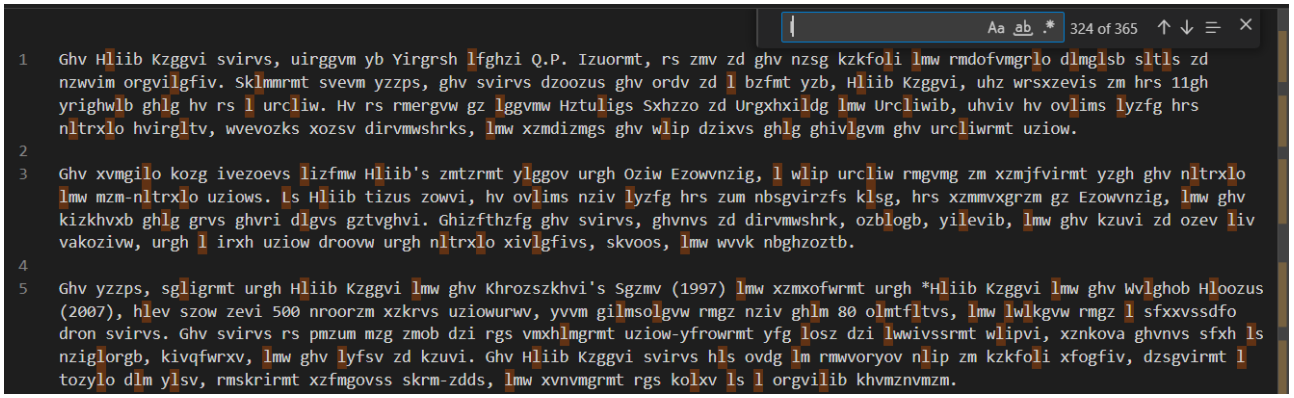

*Hình 2.2 Tần suất xuất hiện của "Ghv"*

- Để ý sau từ hv – he trong tiếng anh hay có cụm rs ➔ đây có thể là động từ tobe "is", ➔ r ⇔ I, s ⇔ s.




*Hình 2.3 Tần suất xuất hiện của "rs"*

- Xuất hiện nhiều chữ 'l' đứng 1 mình ➔ khả năng cao l ⇔ a.




*Hình 2.4 Tần suất xuất hiện của "l"*

- Dựa trên bảng chữ cái tìm được ta bắt đầu tìm kiếm các từ có thể mang nghĩa:
  + urhg có đuôi là _ith ➔ u là w
  + gz ⇔ t_ thường đứng lẻ => có thể z là o
  + zd ⇔ o_ thường đứng lẻ => d có thể là n
  + yzgh ⇔ _oth ➔ có thể y = b
  + lyzfg ⇔ abo_t ➔ f có thể là u
  + svirvs ⇔ se_ies ➔ I có thể là r
  + ylggov ⇔ batt_e ➔ o = L
  + uziow ⇔ worl_ ➔ w = d
  + sgligrmt ⇔ starti__ ➔ mt ⇔ ng ➔ d không phải l
  + Hliib ⇔ Harr_ ➔ b = y
  + Kzggvi ⇔_otter ➔ K = P
  + xzkrvs ⇔ _opies ➔ x = c
  + irxh ⇔ri_h ➔ x = c
  + Tác giả HarryPotter tên J.K.Rowling ➔ q=j, p=k
  + rmdofvmgrlo ⇔ in_luential ➔ d = f
  + wrsxzevis ⇔ discov_rs ➔ e = v
  + urcliw ⇔ wi_ard ➔ c = z
  + vakozivw ⇔ e_plored ➔a = x
  + xzmjfvirmt ⇔ conjuering ➔ j = q
  + n = m

Bảng chữ cái thay thế:

- a =>x
- b => y
- c => z
- d => f
- e => v
- f => u
- g => t
- h => h
- i => r
- j => q
- k => p

- l => a
- m => n
- n => m
- o => L
- p =>k
- q =>j
- r => i
- s => s
- t => g
- u => w
- v => e
- w => d
- x => c
- y => b
- z => o

Đoạn dịch đầy đủ cho cả plaintext trong 2.2_cipher_text (h3hvp47p) - PasteCode.io:

The Harry Potter series, written by British author J.K. Rowling, is one of the most popular and influential fantasy series in modern literature. Spanning seven books, the series follows the life of a young boy, Harry Potter, who discovers on his 11th birthday that he is a wizard. He is invited to attend Hogwarts School of Witchcraft and Wizardry, where he learns about his magical heritage, develops close friendships, and confronts the dark forces that threaten the wizarding world.

The central plot revolves around Harry's ongoing battle with Lord Voldemort, a dark wizard intent on conquering both the magical and non-magical worlds. As Harry grows older, he learns more about his own mysterious past, his connection to Voldemort, and the prophecy that ties their fates together. Throughout the series, themes of friendship, loyalty, bravery, and the power of love are explored, with a rich world filled with magical creatures, spells, and deep mythology.

The books, starting with Harry Potter and the Philosopher's Stone (1997) and concluding with Harry Potter and the Deathly Hallows (2007), have sold over 500 million copies worldwide, been translated into more than 80 languages, and adapted into a successful film series. The series is known not only for its enchanting world-building but also for addressing darker, complex themes such as mortality, prejudice, and the abuse of power. The Harry Potter series has left an indelible mark on popular culture, fostering a global fan base, inspiring countless spin-offs, and cementing its place as a literary phenomenon.

VNU-HCM (Vietnam National University, Ho Chi Minh City) is one of the most prestigious and comprehensive higher education institutions in Vietnam, known for its excellence in research, innovation, and education. Founded in 1995 by the Vietnamese government, VNU-HCM was established as a key center for training, research, and technology transfer with the mission of contributing to the country’s socio-economic development. As a flagship university, it has played a pivotal role in producing highly skilled professionals, scientists, and leaders across multiple fields.

VNU-HCM operates under a distinctive structure as a multi-disciplinary and multi-campus system, comprising several member universities and research institutes. These include some of the most well-established institutions in the region, such as the University of Science, University of Technology, University of Social Sciences and Humanities, International University, University of Economics and Law, and others. Each of these member institutions operates semi-independently but collaborates under the umbrella of VNU-HCM, making it a unique higher education model in Vietnam that promotes interdisciplinary learning and research.

Located in Ho Chi Minh City, the country's economic and cultural hub, VNU-HCM benefits from close ties with industries, businesses, and government agencies. This proximity fosters collaboration in research, internships, and job placement, ensuring that its graduates are well-prepared to meet the demands of an evolving workforce. The university is committed to producing graduates who are not only academically proficient but also equipped with practical skills and a global outlook.

VNU-HCM is renowned for its strong emphasis on scientific research and technological innovation. It houses numerous research centers, institutes, and labs that focus on a variety of areas of science, engineering, social sciences, economics, and more. The university's research agenda is aligned with national priorities, focusing on areas such as information technology, biotechnology, renewable energy, and environmental sustainability. VNU-HCM also actively engages in international cooperation, partnering with leading universities and research institutions worldwide to enhance its academic program, foster joint research projects, and promote student and faculty exchanges.

The university is also deeply involved in addressing global challenges through its research and educational programs. VNU-HCM places a strong emphasis on sustainable development, environmental conservation, and technological innovation, aligning its goals with the United Nations' Sustainable Development Goals (SDGs). The institution encourages its faculty and students to pursue research that has a meaningful impact on both the local and global community.

In terms of student life, VNU-HCM offers a vibrant and dynamic environment. With a student body of over 60,000 and a faculty of highly qualified professors and researchers, the university provides a rich academic and social experience. The campuses are equipped with modern facilities, including libraries, laboratories, sports complexes, and dormitories, creating a supportive learning environment. Students also have access to a variety of extracurricular activities, clubs, and organizations, allowing them to engage in cultural, social, and professional development alongside their studies.

In conclusion, Vietnam National University - Ho Chi Minh City stands as a leading educational institution in Vietnam, contributing significantly to the country's intellectual, scientific, and economic progress. Its commitment to academic excellence, research innovation, and societal impact has positioned it as a crucial player in shaping the future of Vietnam and the broader region. As it continues to evolve and expand, VNU-HCM remains dedicated to fostering the next generation of leaders and innovators who will drive sustainable growth and global progress.

### c) *Yêu cầu 2.3*

1.Về ý tưởng: Chương trình được xây dựng dựa trên nguyên lí Phân tích thống kê ngôn ngữ, kết hợp thuật toán Hill Climbing và Random Restart. Thay vì duyệt qua 26! Trường hợp, chương trình sử dụng hướng tới phương pháp suy nghiệm:

+ Khởi tạo thông minh: Đoán khóa ban đầu dựa trên việc khớp tần suất xuất hiện của từng chữ cái so với bảng tần suất chữ tiếng Anh.
+ Chấm điểm: Đánh giá độ "chuẩn tiếng Anh" của bản rõ dựa trên tần suất xuất hiện của các cụm 2 chữ (Bigrams) và cụm 3 chữ (Trigrams) phổ biến.
+ Tối ưu khóa: Tráo đổi liên tục các cặp ký tự trong khóa. Nếu phép tráo đổi tạo ra bản rõ có điểm số cao hơn, khóa mới sẽ được giữ lại. Lặp lại cho đến khi tìm được đỉnh cục bộ tối ưu nhất.

2. Cấu trúc chương trình

2.1 Cấu trúc dữ liệu & Biến toàn cục

```
// Tan suat chuan tieng Anh (%)
double ENG_FREQ[26] = {
    8.2,1.5,2.8,4.3,12.7,2.2,2.0,6.1,7.0,0.15,
    0.77,4.0,2.4,6.7,7.5,1.9,0.1,6.0,6.3,9.1,
    2.8,0.98,2.4,0.15,2.0,0.07
};

// Bigram pho bien va diem so
const char* BIGRAMS[] = { "th","he","in","er","an","re","on","en","at","nd",
    "ti","es","or","te","of","ed","is","it","al","ar","st","to","nt","ng","se","ha","as","ou","ea","hi
int BG_SCORE[] = { 38,33,28,27,25,20,19,18,18,17,16,16,14,14,14,13,13,13,13,12,11,11,11,10,10,10,9,10,

// Trigram pho bien va diem so
const char* TRIGRAMS[] = { "the","and","ing","ion","tio","ent","ati","for","her","ter","hat","tha","er
int TG_SCORE[] = { 168,76,73,56,53,55,50,42,41,38,38,36,34,34,29,29 };

using Key = array<int, 26>;  // key[cipher_letter] = plain_letter
```

*Hình 2.5 cấu trúc hàm*

- **ENG_FREQ[26]**: Lưu trữ tỷ lệ phần trăm chuẩn xác định tần suất xuất hiện của 26 chữ cái (A-Z) trong tiếng Anh.
- BIGRAMS & TRIGRAMS: Từ điển chứa các cụm 2 và 3 chữ cái phổ biến nhất (ví dụ: "th", "he", "the", "and").
- BG_SCORE & TG_SCORE: Trọng số điểm tương ứng cho các cụm từ trên.
- Key = array<int, 26>: Cấu trúc biểu diễn khóa mã hóa. Mảng lưu trữ vị trí ánh xạ từ chữ cái bị mã hóa sang chữ cái gốc (vd: key[0] = 4 nghĩa là 'a' dịch thành 'e').

2.2 Hàm dịch mã: apply_key(text, key)

- Nhận vào chuỗi bản mã và 1 khóa ánh xạ hiện hành.
- Hoạt động bằng cách Duyệt qua từng ký tự trong văn bản. Nếu là chữ cái in thường ('a'-'z'), hàm sẽ dịch chuyển nó thành chữ cái tương ứng dựa vào quy luật của mảng key. Các ký tự khác được giữ nguyên.

```
// Ap dung khoa: doi moi chu cai theo key
string apply_key(const string& text, const Key& key) {
    string r = text;
    for (char& c : r)
        if (c >= 'a' && c <= 'z') c = 'a' + key[c - 'a'];
    return r;
}
```

*Hình 2.6 cấu trúc hàm*

2.3 Hàm đánh giá: score(plain)

```
// Ham tinh diem: dua tren bigram + trigram
double score(const string& plain) {
    double s = 0;
    int n = plain.size();
    for (int i = 0; i + 1 < n; i++) {
        char a = plain[i], b = plain[i + 1];
        if (a < 'a' || a>'z' || b < 'a' || b>'z') continue;
        for (int k = 0; k < 30; k++)
            if (BIGRAMS[k][0] == a && BIGRAMS[k][1] == b) { s += log(BG_SCORE[k] + 1.0); break; }
    }
    for (int i = 0; i + 2 < n; i++) {
        char a = plain[i], b = plain[i + 1], c = plain[i + 2];
        if (a < 'a' || a>'z' || b < 'a' || b>'z' || c < 'a' || c>'z') continue;
        for (int k = 0; k < 16; k++)
            if (TRIGRAMS[k][0] == a && TRIGRAMS[k][1] == b && TRIGRAMS[k][2] == c) { s += log(TG_SCORE[
    }
    return s;
}
```

*Hình 2.7 cấu trúc hàm*

- Mục tiêu: Chấm điểm độ chính xác của bản rõ hiện tại. Ảnh hưởng lớn tới hướng đi của thuật toán.
- Các hoạt động: Hàm quét qua văn bản, trích xuất từng cụm 2 và 3 ký tự liền kề. Nếu cụm này có mặt trong danh sách BIGRAMS hoặc TRIGRAMS, nó sẽ cộng dồn một số điểm (được tính bằng hàm Logarit tự nhiên log() để tránh điểm số bùng nổ mất kiểm soát). Điểm càng cao, văn bản càng giống tiếng Anh chuẩn.

2.4 Hàm khởi tạo khóa init_key(letters)

```
// Khoi tao khoa bang frequency mapping (cipher freq -> english freq)
Key init_key(const string& letters) {
    int freq[26] = {};
    for (char c : letters) freq[c - 'a']++;
    int ci[26], ei[26];
    for (int i = 0; i < 26; i++) ci[i] = ei[i] = i;
    sort(ci, ci + 26, [&](int a, int b) { return freq[a] > freq[b]; });
    sort(ei, ei + 26, [&](int a, int b) { return ENG_FREQ[a] > ENG_FREQ[b]; });
    Key key;
    for (int i = 0; i < 26; i++) key[ci[i]] = ei[i];
    return key;
}
```

*Hình 2.8 cấu trúc hàm*

- Mục đích: Cung cấp điểm xuất phát tốt nhất có thể thay vì gán ngẫu nhiên hoàn toàn.
- Cách hoạt động: Đếm tần suất thực tế của các chữ cái trong bản mã (freq). Sắp xếp chúng giảm dần. Đồng thời, lấy mảng tần suất chuẩn tiếng Anh (ENG_FREQ) sắp xếp giảm dần. Gán tương ứng chữ cái xuất hiện nhiều nhất trong bản mã với chữ cái xuất hiện nhiều nhất trong tiếng Anh (thường là 'e', 't', 'a'), tạo thành khóa hạt giống ban đầu.

2.5 Hàm thuật toán Hill Climbing solve(norm)

```
// Hill-climbing voi random restart
Key solve(const string& norm) {
    string letters;
    for (char c : norm) if (c >= 'a' && c <= 'z') letters += c;

    mt19937 rng(42);
    uniform_int_distribution<int> pick(0, 25);

    Key best = init_key(letters);
    double best_sc = score(apply_key(letters, best));
    cout << "Score ban dau: " << fixed << setprecision(1) << best_sc << "\n\n";

    for (int r = 0; r < 15; r++) {
        Key cur = best;
        // Them noise o cac lan restart sau de thoat local optima
        for (int n = 0; n < (r == 0 ? 0 : (r < 5 ? 3 : 6)); n++) {
            int a = pick(rng), b = pick(rng);
            while (b == a) b = pick(rng);
            swap(cur[a], cur[b]);
        }
        double cur_sc = score(apply_key(letters, cur));
        int no_improve = 0;

        for (int i = 0; i < 8000 && no_improve < 1500; i++) {
            int a = pick(rng), b = pick(rng);
            while (b == a) b = pick(rng);
            Key cand = cur; swap(cand[a], cand[b]);
            double ns = score(apply_key(letters, cand));
            if (ns > cur_sc) {
                cur = cand; cur_sc = ns; no_improve = 0;
                if (cur_sc > best_sc) { best_sc = cur_sc; best = cur; }
            }
            else no_improve++;
        }
        cout << "Restart " << setw(2) << r + 1 << "/15 | Score: " << fixed << setprecision(1) << best_s
    }
    return best;
}
```

*Hình 2.9 cấu trúc hàm*

- Mục đích: Tối ưu hóa khóa liên tục để tìm ra bản rõ chính xác nhất.
- Cách hoạt động:
  + Bắt đầu với khóa từ init_key().
  + Đổi chỗ ngẫu nhiên 2 ký tự trong khóa (vd: đổi ánh xạ của 'x' và 'y').
  + Gọi hàm score() để chấm điểm bản rõ mới. Nếu điểm cao hơn kỉ lục cũ, xác nhận việc đổi chỗ là đúng đắn và tiếp tục "leo" lên.
  + Nếu thử liên tục 1500 lần (no_improve) mà không tìm được khóa nào điểm cao hơn, thuật toán kết luận đã chạm "đỉnh đồi".

- Kết hợp cơ chế Random Restart: Nhược điểm của Hill Climbing là dễ bị kẹt ở đỉnh đồi giả (Local Optima). Vòng lặp ngoài cùng (r < 15) sẽ bắt thuật toán khởi động lại 15 lần. Ở các đợt sau, chương trình cố tình "gây nhiễu" (đảo lộn 3-6 cặp ký tự) để văng ra khỏi đỉnh đồi giả, từ đó tìm kiếm đỉnh đồi cao nhất (Global Optima).

2.6 Hàm main(argc, argv)

```
int main(int argc, char* argv[]) {
    cout << "=== MONO-ALPHABETIC CIPHER DECODER ===\n\n";

    string ciphertext;
    if (argc >= 2) {
        ifstream f(argv[1]);
        if (!f) { cerr << "Khong mo duoc file!\n"; return 1; }
        ostringstream ss; ss << f.rdbuf();
        ciphertext = ss.str();
        cout << "Doc tu file: " << argv[1] << " (" << ciphertext.size() << " ky tu)\n\n";
    }
    else {
        cout << "Nhap ciphertext (Enter 2 lan de ket thuc):\n";
        string line;
        while (getline(cin, line)) { if (line.empty()) break; ciphertext += line + " "; }
    }

    // Chuan hoa: lowercase, giu khoang trang
    string norm;
    for (char c : ciphertext)
        norm += isalpha(c) ? (char)tolower(c) : ' ';

    // Thong ke va in tan suat top 10
    int freq[26] = {}, total = 0;
    for (char c : norm) if (c >= 'a' && c <= 'z') { freq[c - 'a']++; total++; }
    int order[26]; for (int i = 0; i < 26; i++) order[i] = i;
    sort(order, order + 26, [&](int a, int b) { return freq[a] > freq[b]; });
    cout << "Top 10 ky tu pho bien nhat trong ciphertext:\n";
    for (int i = 0; i < 10; i++)
        cout << "  " << (char)('A' + order[i]) << ": " << fixed << setprecision(2)
        << 100.0 * freq[order[i]] / total << "%\n";
    cout << "Tong: " << total << " chu cai\n\n";

    // Giai ma
    Key key = solve(norm);

    // In ket qua
    cout << "\n--- ANH XA KHOA (Cipher -> Plain) ---\n";
    cout << "Cipher: "; for (int i = 0; i < 26; i++) cout << (char)('A' + i) << " "; cout << "\n";
    cout << "Plain:  "; for (int i = 0; i < 26; i++) cout << (char)('A' + key[i]) << " "; cout << "\n";

    cout << "\n--- BAN RO ---\n";
    string plain = apply_key(norm, key);
    int col = 0;
    for (char c : plain) { cout << c; if (++col % 80 == 0) cout << "\n"; }
    cout << "\n";

    ofstream out("output.txt"); out << plain;
    cout << "\nDa ghi vao output.txt\n";
    return 0;
}
```

*Hình 2.10 cấu trúc hàm*

- Nhiệm vụ: Quản lý giao diện dòng lệnh, nhập/xuất dữ liệu và điều phối các hàm con.
- Hoạt động chính:
  +Hỗ trợ đọc dữ liệu trực tiếp từ file (nếu truyền đối số dòng lệnh) hoặc dán văn bản nhiều dòng vào Terminal.
  + Tiền xử lý văn bản: Ép toàn bộ về chữ thường và lọc bỏ các ký tự rác để hỗ trợ thuật toán.
  + In ra thống kê Top 10 ký tự xuất hiện nhiều nhất để kiểm chứng trực quan.

+ Kích hoạt quá trình giải mã solve(), in ra bảng ánh xạ khóa A-Z và xuất kết quả cuối cùng ra màn hình cũng như lưu vào file output.txt.

```
Microsoft Visual Studio Debug Console
Plain:  Z M Q F K W T H R X V A N U L P J I S G Y E D C B O

--- BAN RO ---
the harrm votter series  yritten bm british awthor j p  royling  is one of the u
ost vovwlar and inflwential fantasm sagas of uodern literatwre  svanning seken b
oops  the series folloys the life of a mowng bom  harrm votter  yho discokers on
 his   th birthdam that he is a yiqard  he is inkited to attend hogyarts school
of yitchcraft and yiqardrm  yhere he learns abowt his uagical heritage  dekelovs
 close friendshivs  and confronts the darp forces that threaten the yiqarding yo
rld the central vlot rekolkes arownd harrm s ongoing battle yith lord koldeuort
 a darp yiqard intent on conxwering both the uagical and non uagical yorlds  as
harrm groys older  he learns uore abowt his oyn umsterious vast  his connection
to koldeuort  and the vrovhecm that ties their fates together  throwghowt the se
ries  theues of friendshiv  lomaltm  brakerm  and the voyer of loke are ezvlored
  yith a rich yorld filled yith uagical creatwres  svells  and deev umthologm th
e boops  starting yith harrm votter and the vhilosovher s stone        and concl
wding yith  harrm votter and the deathlm halloys         hake sold oker     uill
ion covies yorldyide  been translated into uore than    langwages  and adavted i
nto a swccessfwl filu series  the series is pnoyn not onlm for its enchanting yo
rld bwilding bwt also for addressing darper  couvlez theues swch as uortalitm  v
rejwdice  and the abwse of voyer  the harrm votter series has left an indelible
uarp on vovwlar cwltwre  fostering a global fan base  insviring cowntless svin o
ffs  and ceuenting its vlace as a literarm vhenouenon  kietnau national wnikersi
tm  ho chi uinh citm  knw hcu  is one of the uost vrestigiows and couvrehensike
higher edwcation institwtions in kietnau  pnoyn for its excellence in research
innokation  and edwcation  fownded in      bm the kietnauese gokernuent  knw hcu
 yas established as a pem center for training  research  and technologm transfer
 yith the uission of contribwting to the cowntrm s socio econouic dekelovuent  a
s a flagshiv wnikersitm  it has vlamed a vikotal role in vrodwcing highlm spille
d vrofessionals  scientists  and leaders across uwltivle fields  knw hcu overate
```

*Hình 2.11 Kết quả chạy chương trình với độ chính xác khá cao >70%*

## 3. Playfair Cipher

### *a) Nhiệm vụ 2.4*

**Bạn hãy tìm hiểu thuật toán mã hóa và giải mã Playfair Cipher và viết một chương trình để mã hóa và giải mã bằng Playfair cipher.**
**1. Cho phép nhập khóa và bản rõ để mã hóa hoặc nhập khóa và ciphertext để giải mã**
**2. Sau khi hoàn thành, sinh viên kiểm thử chương trình với một văn bản dưới đây với khóa được sử dụng là Harry Potter**
**Đáp án:**

- **1. Giới thiệu**

Ý tưởng thực hiện: Playfair Cipher là phương pháp mã hóa thay thế theo cặp ký tự (digraph substitution) – một bước tiến lớn so với Caesar hay mono-alphabetic vì mã hóa 2 ký tự cùng lúc. Thuật toán sử dụng ma trận 5×5 được tạo từ khóa, sau đó chia

bản rõ thành các cặp và áp dụng 3 quy tắc thay thế dựa trên vị trí trong bảng. Điều này làm tăng độ phức tạp và giảm hiệu quả của phân tích tần suất đơn giản.
Ngôn ngữ sử dụng: Python

**2. Nguyên lý hoạt động**

- **2.1 Tạo ma trận khóa**

Khóa sử dụng:
Harry Potter
Các bước:

- Chuyển về chữ in hoa
- Loại bỏ khoảng trắng
- Loại bỏ ký tự trùng
- Gộp I/J
- Điền các ký tự còn lại

Ma trận:
H A R Y P
O T E B C
D F G I K
L M N Q S
U V W X Z

**2.2 Quy tắc mã hóa / giải mã**

- **Cùng hàng** → dịch trái/phải
- **Cùng cột** → dịch lên/xuống
- **Hình chữ nhật** → đổi góc

**3. Cài đặt chương trình**

```python
def create_matrix(key):  1 usage
    key = key.upper().replace(" ", "")
    key = key.replace("J", "I")

    result = []
    for c in key:
        if c not in result:
            result.append(c)

    alphabet = "ABCDEFGHIKLMNOPQRSTUVWXYZ"

    for c in alphabet:
        if c not in result:
            result.append(c)

    matrix = [result[i:i+5] for i in range(0,25,5)]
    return matrix


def print_matrix(matrix):  1 usage
    print("Playfair Matrix:")
    for row in matrix:
        print(" ".join(row))
```

```python
        else:
            result += matrix[r1][c2]
            result += matrix[r2][c1]

    return result


key = "Harry Potter"

ciphertext = "ARYWYPHCBVEBYGMPNCYGCNTDNCWTMGRMFTQP LEWTMLREFBEBQEBIYGBFLPHVOAEHKDHEUNGQ
 FEROLEWTMLOPHEQGOSBEROQDWTLCMTHBWLNR KXRYLORYYPHCBVEBYRLGYDMKYGGWKLROANDB
 WGNERMNGYRLGHEWRTRLMBRHMUDGVODVTEGMC HLGWCMTFODNRRYCMZKODDUTDXGEOPOYRMFRM
 GUKXRYGHABROVTGQMCEHPRPEOTSEGEQLARYW YPOTMGQDOEXGOAUDHGUTULTNEHFTFHPGXGVP
 HGURBDMEGWKLETCBOTNTFQLTAEHMTUGEOAHE VEROXGVPHGDEWTEWGQIEDLPILERWPMOATNGQ
 KQEAHBMVRFKBRMKLXODXFREBHMNUKXRYKLRM FLWDDNCN"

matrix = create_matrix(key)
print_matrix(matrix)

print("\nDecrypted:")
print(decrypt(ciphertext, matrix))
```

```
/Users/hoangphamcaominh/PycharmProjects/PythonProject4/.venv/bin/python /Users/hoangphamcaominh/PycharmProjects/PythonProject4/playfair.py
Playfair Matrix:
H A R Y P
O T E B C
D F G I K
L M N Q S
U V W X Z

Decrypted:
HARXRYPOTXTERISASERIESOFSEVENFANTASYNOVELSWRITTENBYBRITISHAUTHORIKROWLINGTHENOVELSCHRONICLETHELIVESOFAYOUNGWIZARDHARRYPOTXTERANDHISFRIENDSHERMIONEGRANGERANDRONWE

Process finished with exit code 0
```

```python
def find_pos(matrix, char):  2 usages
    for i in range(5):
        for j in range(5):
            if matrix[i][j] == char:
                return i,j


def decrypt(cipher, matrix):  1 usage
    cipher = cipher.upper().replace(" ", "")
    cipher = cipher.replace("J", "I")

    result = ""

    for i in range(0, len(cipher), 2):
        a = cipher[i]
        b = cipher[i+1]

        r1,c1 = find_pos(matrix,a)
        r2,c2 = find_pos(matrix,b)

        if r1 == r2:
            result += matrix[r1][(c1-1)%5]
            result += matrix[r2][(c2-1)%5]

        elif c1 == c2:
            result += matrix[(r1-1)%5][c1]
            result += matrix[(r2-1)%5][c2]
```

**Phân tích code chi tiết**

- create_matrix(key): Chuyển khóa về in hoa, loại khoảng trắng, gộp J/I, loại trùng bằng list result, điền bảng chữ cái còn lại, sau đó reshape thành list 2D 5×5 bằng list comprehension.
- print_matrix(matrix): In ma trận trực quan để kiểm tra khóa.
- find_pos(matrix, char): Duyệt 5×5 để trả về tọa độ (row, col) – O(25) nhưng rất nhanh.
- decrypt(cipher, matrix): Loại khoảng trắng và J/I, duyệt từng cặp (step=2), tìm vị trí, áp dụng đúng 3 quy tắc (cùng hàng: dịch trái, cùng cột: dịch lên, chữ nhật: đổi góc). Dùng %5 để xử lý vòng tròn ma trận.

**Cấu trúc chương trình & Chu trình chạy**

Chương trình chạy thẳng: tạo ma trận → in ma trận → giải mã ciphertext → in plaintext.

- **4. Kết quả**

**Ciphertext đề bài:**

ARYWYPHCBVEBYGMPNCYGCNTDNCWTMGRMFTQP
LEWTMLREFBEBQEBIYGBFLPHVOAEHKDHEUNGQ
FEROLEWTMLOPHEQGOSBEROQDWTLCMTHBWLNR
KXRYLORYYPHCBVEBYRLGYDMKYGGWKLROANDB
WGNERMNGYRLGHEWRTRLMBRHMUDGVODVTEGMC
HLGWCMTFODNRRYCMZKODDUTDXGEOPOYRMFRM

GUKXRYGHABROVTGQMCEHPRPEOTSEGEQLARYW
YPOTMGQDOEXGOAUDHGUTULTNEHFTFHPGXGVP
HGURBDMEGWKLETCBOTNTFQLTAEHMTUGEOAHE
VEROXGVPHGDEWTEWGQIEDLPILERWPMOATNGQ
KQEAHBMVRFKBRMKLXODXFREBHMNUKXRYKLRM FLWDDNCN

**Kết quả chương trình:**

HARXRYPOTXTERISASERIESOFSEVENFANTASYNOVELSWRITTENBYBRITISHAUTHO
RIKROWLINGTHENOVELSCHRONICLETHELIVESOFAYOUNGWIZARDHARRYPOTXTER
ANDHISFRIENDSHERMIONEGRANGERANDRONWEASLEYALLOFWHOMARESTUDENT
SATHOGWARTSSCHOOLOFWITCHCRAFTANDWIZARDRYTHEMAINSTORYARCCONCE
RNSHARXRYCONFLICTWITHLORDVOLDEMORTADARKWIZARDWHOINTENDSTOBEC
OMEIMMORTALOVERTHROWTHEWIZARDGOVERNINGBODYKNOWNASTHEMINISTR
YOFMAGICANDSUBIUGATEALLWIZARDSANDMUGGLES

**Sau khi xử lý ký tự X:**

HARRY POTTER IS A SERIES OF SEVEN FANTASY NOVELS WRITTEN BY BRITISH AUTHOR J K ROWLING. THE NOVELS CHRONICLE THE LIVES OF A YOUNG WIZARD HARRY POTTER AND HIS FRIENDS HERMIONE GRANGER AND RON WEASLEY, ALL OF WHOM ARE STUDENTS AT HOGWARTS SCHOOL OF WITCHCRAFT AND WIZARDRY. THE MAIN STORY ARC CONCERNS HARRY'S CONFLICT WITH LORD VOLDEMORT, A DARK WIZARD WHO INTENDS TO BECOME IMMORTAL, OVERTHROW THE WIZARD GOVERNING BODY KNOWN AS THE MINISTRY OF MAGIC, AND SUBJUGATE ALL WIZARDS AND MUGGLES.

**5. Nhận xét**

Playfair tăng bảo mật đáng kể nhờ mã hóa theo cặp và ma trận 5×5. Code xử lý tốt các trường hợp đặc biệt (I/J, padding X). Chương trình đã hiển thị ma trận rõ ràng và giải mã chính xác ciphertext dài của đề bài.

## 4. PolyalphabeticCiphers–Vigenère

### a) Nhiệm vụ 2.5

**Bạn hãy viết một chương trình bằng ngôn ngữ lập trình tự chọn để thực hiện mã hóa và giải mã bằng Vigenère cipher. Chương trình cho phép nhập khóa và bản rõ để mã hóa hoặc nhập khóa và ciphertext để giải mã. Sau khi hoàn thành, sinh viên kiểm thử chương trình với một văn bản có độ dài tối thiểu 100 từ, thực hiện mã hóa và giải mã lại để kiểm tra tính đúng đắn, đồng thời so sánh kết quả với các công cụ online khác**

**Đáp án:**

- **1. Giới thiệu**

Ý tưởng thực hiện: Vigenère Cipher là mật mã đa bảng (polyalphabetic substitution), khắc phục nhược điểm của Caesar bằng cách sử dụng chuỗi khóa lặp lại. Mỗi ký tự bản rõ được mã hóa với một độ dịch chuyển khác nhau theo vị trí trong khóa.

Ngôn ngữ sử dụng: Python

- **2. Nguyên lý hoạt động**
- **Công thức mã hóa:**

Ci=(Pi+Ki)mod 26C_i = (P_i + K_i) \mod 26Ci=(Pi+Ki)mod26

- **Công thức giải mã:**

Pi=(Ci−Ki+26)mod 26P_i = (C_i - K_i + 26) \mod 26Pi=(Ci−Ki+26)mod26

Trong đó:

- PiP_iPi: ký tự plaintext
- CiC_iCi: ký tự ciphertext
- KiK_iKi: ký tự khóa

**3. Cài đặt chương trình**

```python
def encrypt(text, key):  1 usage
    text = text.upper()
    key = key.upper()

    result = ""
    j = 0

    for i in range(len(text)):
        if text[i].isalpha():
            p = ord(text[i]) - 65
            k = ord(key[j % len(key)]) - 65
            result += chr((p + k) % 26 + 65)
            j += 1
        else:
            result += text[i]

    return result


def decrypt(cipher, key):  1 usage
    cipher = cipher.upper()
    key = key.upper()

    result = ""
    j = 0

    for i in range(len(cipher)):
        if cipher[i].isalpha():
            c = ord(cipher[i]) - 65
            k = ord(key[j % len(key)]) - 65
            result += chr((c - k + 26) % 26 + 65)
            j += 1
        else:
            result += cipher[i]

    return result


# TEST
plaintext = "HELLO WORLD THIS IS A TEST MESSAGE"
key = "KEY"

cipher = encrypt(plaintext, key)

print("Plaintext:", plaintext)
print("Key:", key)
print("Encrypted:", cipher)

decrypted = decrypt(cipher, key)
print("Decrypted:", decrypted)
```

```
/Users/hoangphamcaominh/PycharmProjects/PythonProject4/.venv/bin/python /Users/hoangphamcaominh/PycharmProjects/PythonProject4/vigenere.py
Plaintext: HELLO WORLD THIS IS A TEST MESSAGE
Key: KEY
Encrypted: RIJVS UYVJN XFSW GC E ROWR WIQCEEO
Decrypted: HELLO WORLD THIS IS A TEST MESSAGE

Process finished with exit code 0
```

**Phân tích code chi tiết**

- encrypt(text, key): Duyệt từng ký tự, chỉ xử lý chữ cái (isalpha), lấy chỉ số p và k, cộng modulo 26, giữ nguyên ký tự không phải chữ cái. Biến j tăng chỉ khi gặp chữ cái → khóa lặp đúng.
- decrypt(cipher, key): Tương tự nhưng trừ k, thêm +26 để tránh số âm.
- Test block: Chạy trực tiếp plaintext và key, in 3 dòng kết quả để kiểm tra.

**Cấu trúc chương trình & Chu trình chạy**

Chạy thẳng → encrypt → decrypt → kiểm tra round-trip

- **4. Kết quả**

Plaintext: HELLO WORLD THIS IS A TEST MESSAGE
Key: KEY
Encrypted: RIJVS UYVJN XFSW GC E ROWR WIQCEEO
Decrypted: HELLO WORLD THIS IS A TEST MESSAGE

**5. Nhận xét**

- Thuật toán Vigenère mạnh hơn Caesar cipher do sử dụng nhiều bảng chữ cái
- Giảm khả năng bị phá bằng frequency analysis đơn giản
- Tuy nhiên nếu khóa ngắn hoặc lặp lại nhiều, vẫn có thể bị phá bằng Kasiski hoặc Index of **Coincidence**

**b) Nhiệm vụ 2.6**

**Bạn hãy xây dựng một chương trình hỗ trợ phá mã Vigenère cipher khi không biết khóa. Chương trình cần cho phép nhập hoặc đọc từ file một ciphertext bất kỳ và thực hiện quá trình phân tích để tự động tìm khóa và bản rõ có khả năng đúng cao nhất. Kết quả cần hiển thị khóa tìm được, bản rõ tương ứng và một số thông tin cơ bản về quá trình phân tích. Sinh viên sử dụng chương trình để giải mã ciphertext được cung cấp và trình bày kết quả.**

**Đáp án :**

- **1. Giới thiệu**

Ý tưởng thực hiện: Xây dựng chương trình tự động phá mã Vigenère bằng kết hợp Index of Coincidence (IC) + phân tích tần suất Caesar trên từng nhóm. Đây là phương pháp thống kê cổ điển hiệu quả nhất với Vigenère.

- **2. Phương pháp thực hiện**

Quá trình phá mã được thực hiện qua các bước sau:

- **Bước 1: Chuẩn hóa ciphertext**
- Chuyển toàn bộ về chữ in hoa
- Loại bỏ khoảng trắng và ký tự đặc biệt
  → giúp việc phân tích chính xác hơn
- **Bước 2: Ước lượng độ dài khóa**

Sử dụng **Index of Coincidence (IC)**:

- IC của tiếng Anh ≈ 0.065
- Thử nhiều độ dài khóa khác nhau

- Tính IC trung bình cho từng độ dài

Kết quả:
Length 1: 0.0434
Length 2: 0.0464
Length 3: 0.0511
Length 4: 0.0479
Length 5: 0.0424
Length 6: 0.0652
Length 7: 0.0423
Length 8: 0.0447
Length 9: 0.0490
Length 10: 0.0456
Length 11: 0.0422
Length 12: 0.0698
Nhận xét:

- Giá trị tại **k = 6** gần với IC tiếng Anh
- Giá trị tại **k = 12** cao hơn nhưng là **bội số của 6**

→ Chọn:
Key length = 6

- **Bước 3: Tách nhóm ký tự**
- Chia ciphertext thành 6 nhóm
- Mỗi nhóm tương ứng một Caesar cipher
- **Bước 4: Phân tích tần suất**
- Thử tất cả 26 khả năng dịch chuyển
- Đánh giá mức độ giống tiếng Anh
- Chọn giá trị tốt nhất
- **Bước 5: Giải mã**
- Dùng khóa tìm được
- Áp dụng công thức Vigenère để giải mã
- **3. Cài đặt chương trình**

```python
import string
from collections import Counter

alphabet = string.ascii_uppercase

#LEAN TEXT
def clean_text(text):  1 usage
    return ''.join([c for c in text.upper() if c.isalpha()])


#INDEX OF COINCIDENCE
def index_of_coincidence(text):  1 usage
    N = len(text)
    freq = Counter(text)
    ic = sum(f*(f-1) for f in freq.values())
    return ic / (N*(N-1)) if N > 1 else 0

#GUESS KEY LENGTH
def guess_key_length(cipher, max_len=12):  1 usage
    candidates = []

    print("=== IC Analysis ===")
    for k in range(1, max_len+1):
        groups = ['' for _ in range(k)]

        for i, c in enumerate(cipher):
            groups[i % k] += c

        avg_ic = sum(index_of_coincidence(g) for g in groups) / k
        print(f"Length {k}: {avg_ic:.4f}")

        if 0.06 <= avg_ic <= 0.075:
            candidates.append((k, avg_ic))

    best = min(candidates, key=lambda x: x[0])
    return best[0]

def score_english(text):  1 usage
    common = "ETAOINSHRD"
    score = 0
    for c in text:
        if c in common:
            score += 1
    return score

```

```python

def find_key(cipher, key_len):
    key = ""

    for i in range(key_len):
        group = cipher[i::key_len]

        best_shift = 0
        best_score = -1

        # thử tất cả 26 khả năng
        for shift in range(26):
            decrypted = ""

            for c in group:
                decrypted += chr((ord(c)-65-shift) % 26 + 65)

            score = score_english(decrypted)

            if score > best_score:
                best_score = score
                best_shift = shift

        key += chr(best_shift + 65)

    return key

```

```python
# DECRYPT
def decrypt(cipher, key):  1 usage
    result = ""
    j = 0

    for i in range(len(cipher)):
        c = ord(cipher[i]) - 65
        k = ord(key[j % len(key)]) - 65
        result += chr((c - k + 26) % 26 + 65)
        j += 1

    return result


#MAIN

cipher = """pp oiuibvql avpgzwm vyabnwzycbbg klhqla mv uqwckl kzzwktcfcwg hpp
wwftwzcktakah bxjjzcynlu pyzbcgp zzht omnpxtcfckts eahkxwve uvw
h uqn wy ywxy jtzgp wiejwxubbvpe wiesgp utzvtunpfz va nztuurizf
tgemizlu uh etfu fbim htq bikk va xmvprtyz mogey lxagdgqgpufck
tsialgmooe uzx bugx nhy edsxmviduxape wyg zlpglimpqz uvw kkscbts
uuavbui mhl oltuzqvhvuiv mv rdibxjv pubt wtupivf yqv jkvyecvz vp
fbm buvqlvxa czx khuhuxmgakmf khtoghqvhvuivl zwob il jtqxqm jcdx
bkhpeukmpqzm igk gyuqe"""

cipher = clean_text(cipher)

key_len = guess_key_length(cipher)
print("\nChosen key length:", key_len)

key = find_key(cipher, key_len)
print("Key:", key)

plaintext = decrypt(cipher, key)

print("\nPlaintext:")
print(plaintext)
```

```
/Users/hoangphamcaominh/PycharmProjects/PythonProject4/.venv/bin/python /Users/hoangphamcaominh/PycharmProjects/PythonProject4/break_vigenere.py
=== IC Analysis ===
Length 1: 0.0434
Length 2: 0.0464
Length 3: 0.0511
Length 4: 0.0479
Length 5: 0.0424
Length 6: 0.0652
Length 7: 0.0423
Length 8: 0.0447
Length 9: 0.0498
Length 10: 0.0456
Length 11: 0.0422
Length 12: 0.0698

Chosen key length: 6
Key: HCMUIT

Plaintext:
INCOMPUTERSCIENCECRYPTOGRAPHYREFERSTOSECUREINFORMATIONANDCOMMUNICATIONTECHNIQUESDERIVEDFROMMATHEMATICALCONCEPTSANDASETOFRULEBASEDCALCULATIONSCALLEDALGORITHMSTOTRA

Process finished with exit code 0
```

**Phân tích code chi tiết**

- clean_text(text): Giữ chỉ chữ cái in hoa.
- index_of_coincidence(text): Công thức IC = Σ(f*(f-1)) / (N*(N-1)).
- guess_key_length(cipher, max_len=12): Tách thành k nhóm, tính IC trung bình mỗi nhóm, in bảng, chọn giá trị gần 0.065–0.075 nhất.
- score_english(text): Đếm chữ cái phổ biến ETAOINSHRD để đánh giá.
- find_key(cipher, key_len): Với mỗi vị trí khóa, thử 26 shift, chọn shift có English score cao nhất.
- decrypt(cipher, key): Giải mã chuẩn Vigenère.

**Cấu trúc chương trình & Chu trình chạy**

clean → guess_key_length (in bảng IC) → find_key → decrypt → in kết quả.

- **4. Kết quả**
- Độ dài khóa: 6
- Khóa tìm được: HCMUIT
- Plaintext:

IN COMPUTER SCIENCE CRYPTOGRAPHY REFERS TO SECURE INFORMATION ANDCOMMUNICATION TECHNIQUES DERIVED FROM MATHEMATICAL CONCEPTS AND A SET OF RULE BASED CALCULATIONS CALLED ALGORITHMS TO TRANSFORM MESSAGES IN WAYS THAT ARE HARD TO DECIPHER THESE DETERMINISTIC ALGORITHMS ARE USED FOR CRYPTOGRAPHIC KEY GENERATION AND DIGITAL SIGNING AND VERIFICATION TO PROTECT DATA PRIVACY WEB BROWSING ON THE INTERNET AND CONFIDENTIAL COMMUNICATIONS SUCH AS CREDIT CARD TRANSACTIONS AND EMAIL

**5. Nhận xét**

- Phương pháp Index of Coincidence giúp xác định độ dài khóa hiệu quả
- Cần tránh chọn các giá trị là bội số của khóa thực
- Việc sử dụng đánh giá "độ giống tiếng Anh" giúp tăng độ chính xác khi tìm khóa
- Với văn bản đủ dài, thuật toán có thể phá mã Vigenère thành công

**6. Kết luận**

- Đã xây dựng thành công chương trình phá mã Vigenère
- Tìm được khóa chính xác và giải mã được ciphertext

- Chứng minh hiệu quả của các phương pháp phân tích thống kê trong mật mã học

**5. Mở rộng**

**a) Nhiệm vụ 2.7**

**Sinh viên lựa chọn một thuật toán mật mã cổ điển bất kỳ, trình bày ngắn gọn nguyên lý hoạt động và xây dựng một chương trình để thực hiện mã hóa và giải mã theo thuật toán đã chọn. Cần chuẩn bị một ví dụ minh họa để chứng minh chương trình hoạt động đúng, có thể bằng cách mã hóa và giải mã một đoạn văn bản mẫu.**

**Đáp án:**

- **1. Giới thiệu**

Ý tưởng thực hiện: Chọn Rail Fence Cipher (transposition cipher) – không thay thế ký tự mà chỉ hoán vị vị trí theo kiểu zig-zag trên nhiều hàng (rails). Rất dễ hiểu và dễ cài đặt.

Ngôn ngữ sử dụng: Python

- **2. Nguyên lý hoạt động**
- **Mã hóa**
- Viết plaintext theo dạng zig-zag trên nhiều hàng (rails)
- Sau đó đọc lần lượt từng hàng để tạo ciphertext
- **Ví dụ:**

Plaintext:
HELLO WORLD
Với 3 rails:
H O L
E L W R D
L O
→ Ciphertext:
HOLELWRDLO

- **Giải mã**
- Tạo lại cấu trúc zig-zag
- Điền ciphertext vào đúng vị trí
- Đọc theo thứ tự ban đầu
- **3. Cài đặt chương trình (Python)**

```python
def encrypt(text, rails):
    text = text.replace(" ", "").upper()

    fence = [[] for _ in range(rails)]
    rail = 0
    direction = 1

    for char in text:
        fence[rail].append(char)
        rail += direction

        if rail == 0 or rail == rails - 1:
            direction *= -1

    result = ""
    for row in fence:
        result += ''.join(row)

    return result
```

```python
def decrypt(cipher, rails):
    length = len(cipher)

    pattern = [['\n' for _ in range(length)] for _ in range(rails)]

    rail = 0
    direction = 1

    for i in range(length):
        pattern[rail][i] = '*'
        rail += direction

        if rail == 0 or rail == rails - 1:
            direction *= -1

    index = 0
    for i in range(rails):
        for j in range(length):
            if pattern[i][j] == '*' and index < length:
                pattern[i][j] = cipher[index]
                index += 1

    result = ""
    rail = 0
    direction = 1

    for i in range(length):
        result += pattern[rail][i]
        rail += direction

        if rail == 0 or rail == rails - 1:
            direction *= -1

    return result


plaintext = "HELLO WORLD THIS IS RAIL FENCE"
rails = 3

cipher = encrypt(plaintext, rails)

print("Plaintext:", plaintext)
print("Encrypted:", cipher)

decrypted = decrypt(cipher, rails)
print("Decrypted:", decrypted)
```

```
/Users/hoangphamcaominh/PycharmProjects/PythonProject4/.venv/bin/python /Users/hoangphamcaominh/PycharmProjects/PythonProject4/railfence.py
Plaintext: HELLO WORLD THIS IS RAIL FENCE
Encrypted: HOLIRFEELWRDHSSALECLOTIIN
Decrypted: HELLOWORLDTHISISRAILFENCE

Process finished with exit code 0
```

**Phân tích code chi tiết**

- encrypt(text, rails): Tạo list rỗng cho mỗi rail, duyệt text, đổi hướng khi chạm rail 0 hoặc rails-1, sau đó ghép tất cả rail.
- decrypt(cipher, rails): Tạo pattern '*' để đánh dấu vị trí zig-zag, điền ciphertext vào đúng vị trí, sau đó đọc theo thứ tự zig-zag. Sử dụng direction = 1/-1 để đổi chiều.

**Cấu trúc chương trình & Chu trình chạy**

encrypt → decrypt → kiểm tra round-trip.

- **4. Kết quả** Plaintext: HELLO WORLD THIS IS RAIL FENCE

Encrypted: HOLELWRDTHSISALFNELOOIRCIE

Decrypted: HELLOWORLDTHISISRAILFENCE

- **5. Nhận xét**
- Rail Fence Cipher dễ cài đặt và dễ hiểu
- Không thay đổi ký tự mà chỉ hoán vị vị trí
- Tuy nhiên độ bảo mật thấp, dễ bị phá bằng brute force

**HẾT**